

Offline Deployment Guide

Automation Deployment

1. Copy file bag-doorprize-deploy.tar.gz to folder /home/sysadmin/bagi-hoki-main/YYYY-MM-DD/bag-doorprize-deploy.tar.gz.

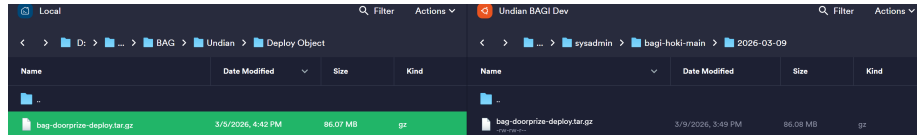


Figure 1: Copy file bag-doorprize-deploy.tar.gz to server

2. Make sure you copy file deploy.sh to /home/sysadmin/ or /home/root/ (Based on project placed).

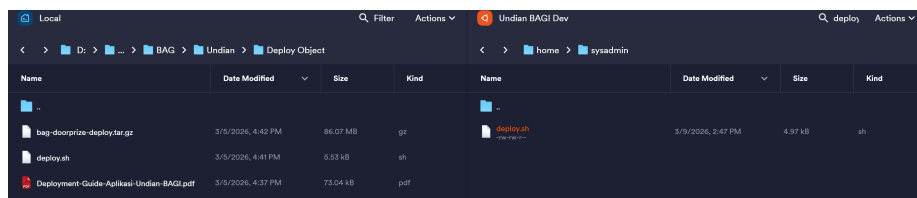


Figure 2: Copy deploy.sh to server

3. Then run this script to make file deploy.sh executable `sudo chmod +x deploy.sh`

```
sysadmin@undian-bagi-dev:~$ sudo chmod +x deploy.sh
[sudo] password for sysadmin:
sysadmin@undian-bagi-dev:~$ ls -larthi | grep deploy.sh
579235 -rwxrwxr-x 1 sysadmin sysadmin 5.0K Mar  9 07:47 deploy.sh
sysadmin@undian-bagi-dev:~$
```

Figure 3: Give permission executable file for deploy.sh

4. Run file deploy.sh by this script `sudo bash deploy.sh`
5. Deployment successfull

```
sysadmin@undian-bagi-dev:~$ sudo bash deploy.sh

[INFO] Starting deployment: bagi-hoki
[INFO] Time: Mon Mar 9 09:48:59 UTC 2026

[OK] Deployment archive found.

[INFO] Step 1/6 – Stopping services...
laravel-octane: stopped
laravel-worker:laravel-worker_00: stopped
laravel-worker:laravel-worker_01: stopped
laravel-worker:laravel-worker_02: stopped
laravel-worker:laravel-worker_03: stopped
laravel-worker:laravel-worker_04: stopped
laravel-worker:laravel-worker_05: stopped
laravel-worker:laravel-worker_06: stopped
laravel-worker:laravel-worker_07: stopped
[OK] Services stopped.

[INFO] Step 2/6 – Creating lightweight backup...
[OK] Backup created:
/home/sysadmin/bagi-hoki-backup/backup-20260309-094859.tar.gz

[INFO] Step 3/6 – Cleaning old deployment...
[OK] Old files removed.

[INFO] Step 4/6 – Extracting new release...
[OK] Extraction completed.
```

Figure 4: Run bash deploy.sh

```
>>> Deployment complete! App is ready.  
[OK]   Application deployment completed.  
  
[INFO] Step 6/6 – Restarting services...  
laravel-octane: started  
laravel-worker:laravel-worker_00: started  
laravel-worker:laravel-worker_01: started  
laravel-worker:laravel-worker_02: started  
laravel-worker:laravel-worker_03: started  
laravel-worker:laravel-worker_04: started  
laravel-worker:laravel-worker_05: started  
laravel-worker:laravel-worker_06: started  
laravel-worker:laravel-worker_07: started  
[OK]   Services restarted.  
  
[INFO] Checking nginx status...  
[OK]   Nginx is running.  
  
✓ DEPLOYMENT SUCCESSFUL  
Time: Mon Mar 9 09:49:07 UTC 2026
```

Figure 5: Successfully running deployment

Additional Deployment For Generate PDF Statement

1. First of all, make sure that you have cleared the uploaded pdf statement on server T24. You can check the uploaded pdf-statement by aplikasi undian by using sql query inside database bagihoki.
 - Login to database (PostgreSQL) using `sudo -u postgres psql`
 - Run this command to change database to bagihoki `\c bagihoki`
 - Run this query `SELECT file_name_t24, file_path_t24 FROM account_documents;` to check the uploaded pdf-statement.
 - Uploaded document pdf-statement will shown, now you just need to access SFTP server and remove those files that listed on `file_path_t24` field.
2. When you have removed the whole files, now you just need to run this command to erase all data on `account_documents`'s table on database bagihoki using this script `truncate account_documents.`
3. After cleansing the database, you have to restart the service using this command `sudo supervisorctl restart all` and `php artisan optimize:clear` to clear all cache in application.
4. Then run this command to generate new pdf and it will uploaded into server SFTP T24 `php artisan app:generate-bank-statement-pdf-command --account_numbers='5310251159','0093217268','0123200314','5320223708','5310237919','108`
5. Check the uploaded files at SFTP T24 or you can directly check the path of uploaded file in database bagihoki using this command `SELECT file_name_t24, file_path_t24 FROM account_documents;`, Note: make sure you have logged in into postgresql and accessing database bagihoki before run this command.
6. Uploading pdf-statement successfully.

Releasing Data Point and Ticket

Data From Big Data

1. Login into database big data through server aplikasi undian using this command `psql -h 10.0.230.64 -p 5432 -U appundian -d foundation_mis`
2. After you logged in into database big data, you need to run these query below:
 - NTB - Bulan Januari
`\copy (SELECT cif, name, ac_id, jenis_rekening, ROUND(avg_balance::numeric, 2) as avg_balance, account_opening_date, acc_open_branch, cus_open_branch, file_date, inactiv_marker, exclude_flag, confi_flag FROM core_t24_temp.undian_ntb WHERE file_date =`

```
'2026-01-31') TO '/tmp/ntb_big_data_2026_01_TO.csv' WITH CSV
HEADER;
```

- ETB - Bulan Januari

```
\copy (SELECT cif, name, ac_id, jenis_rekening, ROUND(avg_balance::numeric,
2) as avg_balance, account_opening_date, acc_open_branch,
cus_open_branch, file_date, inactiv_marker, exclude_flag,
confi_flag FROM core_t24_temp.undian_etb WHERE file_date =
'2026-01-31') TO '/tmp/etb_big_data_2026_01_TO.csv' WITH CSV
HEADER;
```

- NTB - Bulan Februari

```
\copy (SELECT cif, name, ac_id, jenis_rekening, ROUND(avg_balance::numeric,
2) as avg_balance, account_opening_date, acc_open_branch,
cus_open_branch, file_date, inactiv_marker, exclude_flag,
confi_flag FROM core_t24_temp.undian_ntb WHERE file_date =
'2026-02-28') TO '/tmp/ntb_big_data_2026_02_TO.csv' WITH CSV
HEADER;
```

- ETB - Bulan Februari

```
\copy (SELECT cif, name, ac_id, jenis_rekening, ROUND(avg_balance::numeric,
2) as avg_balance, account_opening_date, acc_open_branch,
cus_open_branch, file_date, inactiv_marker, exclude_flag,
confi_flag FROM core_t24_temp.undian_etb WHERE file_date =
'2026-02-28') TO '/tmp/etb_big_data_2026_02_TO.csv' WITH CSV
HEADER;
```

3. Data query from big data will shown at /tmp folders, run this command to show it `ls /tmp | grep big_data`.
4. After you have run the query, you need to move the file to /home/sysadmin/ or /home/root/ (Based on project placed) and download it to your local computer.

Data From Aplikasi Undian

1. Login into database big data through server aplikasi undian using this command `psql -U bagihoki -d bagihoki`
 2. After you logged in into database big data, you need to run these query below:
- All Data

```
\copy (SELECT c.cif, a.account_number, c.name, ph.points,
lt.range_start || ' - ' || lt.range_end AS range_ticket,
ph.amount AS avg_bal, a.account_opening_balance, a.account_opening_date,
lt.month, lt.year FROM customers c JOIN accounts a ON c.id =
```

```
a.customer_id JOIN point_histories ph ON a.id = ph.account_id
LEFT JOIN participants p ON a.id = p.account_id LEFT JOIN
lottery_tickets lt ON p.id = lt.participant_id AND lt.month
= ph.month AND lt.year = ph.year WHERE c.deleted_at IS NULL
AND a.deleted_at IS NULL AND ph.deleted_at IS NULL ORDER BY
ph.year DESC, ph.month DESC) TO '/tmp/data_undian_TO.csv'
WITH CSV HEADER
```

3. Data query from big data will shown at /tmp folders, run this command to show it `ls /tmp | grep data_undian`.
4. After you have run the query, you need to move the file to /home/sysadmin/ or /home/root/ (Based on project placed) and download it to your local computer.

Notes:

If you have implemented automation deploy, preparation below will be optional.

Manual Deployment

This guide explains how to deploy the application to a server with **no internet connection**.

1. Local Preparation (With Internet)

Run these steps on your development machine where you have internet access.

1. Open your terminal in the project root.
2. Execute the following command to build and package the app: `bash make package`
3. This will:
 - Install production dependencies (`vendor/`).
 - Build frontend assets (`public/build/`).
 - Compact everything into a file named `bag-doorprize-deploy.tar.gz`.

2. Server Setup (Offline)

Move the generated `bag-doorprize-deploy.tar.gz` to your target server using a USB drive or local network.

1. Extract the package: `bash tar -xzf bag-doorprize-deploy.tar.gz -C /path/to/webroot`
2. Navigate to the project directory: `bash cd /path/to/webroot`
3. Create and configure your `.env` file manually: `bash cp .env.example .env nano .env # Adjust DB_HOST, DB_PASSWORD, etc. # Set OCTANE_SERVER=frankenphp`
4. Run the deployment script: `bash make deploy`

5. Setup the automated task scheduler (Crontab): `bash make cron`

3. Running the Application (Octane + FrankenPHP)

This application is designed to run with **Laravel Octane** and **FrankenPHP**.

To start the application server:

```
make octane
```

The server will start on port 8000 by default (as defined in the Makefile).

4. Setup User Management & Permissions

Create Super Admin User

Create a super admin user using this command:

```
php artisan shield:super-admin
```

This will prompt you for:

- Name
- Email
- Password

The command will create a user with the `super_admin` role and all permissions.

Install Shield Filament Generator (Optional - For Development)

If you need to regenerate Shield resources or customize permissions, install the Shield generator:

1. **Install the package** (requires internet):

```
composer require bezhansalleh/filament-shield --dev
```

2. **Publish Shield resources:**

```
php artisan vendor:publish --tag="filament-shield-config"
```

3. **Generate Shield resources** (roles, permissions UI):

```
php artisan shield:install
```

This will:

- Create migrations for roles and permissions
- Generate Shield resources (RoleResource, etc.)
- Set up permission policies

4. **Generate permissions for existing resources:**

```
php artisan shield:generate --all
```

This scans all Filament resources and creates permissions for them.

5. **Create custom roles** (if needed): `bash php artisan shield:create-role`

Shield Commands Reference

Command	Description
<code>php artisan shield:super-admin</code>	Create a super admin user
<code>php artisan shield:install</code>	Install Shield resources
<code>php artisan shield:generate --all</code>	Generate permissions for all resources
<code>php artisan shield:generate --resource=CustomerResource</code>	Generate permissions for specific resource
<code>php artisan shield:create-role</code>	Create a new role interactively
<code>php artisan shield:publish</code>	Publish Shield views and config

Managing Permissions

After installation, you can manage roles and permissions through the Filament admin panel:

1. Navigate to `/admin/shield/roles`
2. Create/edit roles
3. Assign permissions to roles
4. Assign roles to users

Notes

- **PHP & Web Server:** Ensure the server already has PHP 8.2+, a database (PostgreSQL/MySQL), and required extensions. FrankenPHP handles web requests directly, so a separate Nginx/Apache is optional.
- **Binary Status:** The `frankenphp` binary is bundled within the deployment package. The `make deploy` command automatically sets the correct execution permissions for it.